

AI-powered bioinformatic analysis using R, #3

R Language Basics II — Factors, Lists, Data Frames & File I/O

Wei-Hua Chen (CC BY-NC 4.0)

HUST, China

2026-05-27

Table of contents I

- 1 Factors
- 2 Lists
- 3 Data Frames & Tibbles
- 4 Data Import & Export
- 5 Tidy Data
- 6 Dates with lubridate
- 7 Summary & Homework



What is a Factor?

In bioinformatics, many variables are **categorical**: - Treatment groups: “Control”, “Drug A”, “Drug B” - Tissue types: “Liver”, “Kidney”, “Brain” - Genotypes: “AA”, “Aa”, “aa” - Disease status: “Healthy”, “Mild”, “Severe”

Factors are R’s way of handling categorical data. They store values as **integers** with a set of **labels** (levels).



Tip

Factors look like characters but behave like **coded integers** — this is crucial for statistical modeling!

Creating Factors

Use `factor()` to create a factor from a vector:

```
# A character vector of treatment groups
treatment <- c("Control", "Drug", "Control", "Drug", "Control")

# Convert to factor
( treat_f <- factor(treatment) )
```

```
[1] Control Drug    Control Drug    Control
Levels: Control Drug
```

Creating Factors, cont.

Check the internal structure:

```
class(treat_f)
```

```
[1] "factor"
```

```
str(treat_f)      # shows integer codes + levels
```

```
Factor w/ 2 levels "Control","Drug": 1 2 1 2 1
```

```
levels(treat_f)   # unique categories
```

```
[1] "Control" "Drug"
```

Factor Levels — Default Order

By default, levels are ordered **alphabetically**:

```
group <- c("High", "Low", "Medium", "Low", "High")  
factor(group)
```

```
[1] High   Low    Medium Low    High  
Levels: High Low Medium
```

```
# Levels are: High, Low, Medium (alphabetical)  
# This may NOT be what you want for plotting or modeling!
```

For plots and statistical models, you often need a **meaningful order**: Low < Medium < High (not alphabetical).

Controlling Level Order

Specify the levels argument explicitly:

```
( group_f <- factor(group,  
  levels = c("Low", "Medium", "High")) )
```

```
[1] High   Low    Medium Low    High  
Levels: Low Medium High
```

```
# Now levels are in the correct logical order  
levels(group_f)
```

```
[1] "Low"      "Medium"   "High"
```

Ordered Factors

Use `ordered = TRUE` (or `ordered()`) to create an **ordered factor**:

```
( severity <- ordered(c("Mild", "Severe", "Moderate", "Mild"),
                      levels = c("Mild", "Moderate", "Severe")) )
```

```
[1] Mild      Severe    Moderate Mild
Levels: Mild < Moderate < Severe
```

```
# Comparisons work on ordered factors
severity[1] < severity[2]      # "Mild" < "Severe" → TRUE
```

```
[1] TRUE
```

Warning

Only use `ordered = TRUE` when the categories have a **genuine** order (e.g., disease stage).
Don't order nominal categories like "Red", "Green", "Blue".

Factor Manipulation with forcats

The **forcats** package (part of tidyverse) provides tools for factor management:

```
library(forcats) # or library(tidyverse)
```

```
# fct_relevel() — reorder levels manually
fct_relevel(treat_f, "Drug", "Control")
```

```
[1] Control Drug    Control Drug    Control
Levels: Drug Control
```

```
# fct_rev() — reverse level order
fct_rev(treat_f)
```

```
[1] Control Drug    Control Drug    Control
Levels: Drug Control
```

```
# fct_infreq() — order by frequency
fct_infreq(treat_f)
```

```
[1] Control Drug    Control Drug    Control
Levels: Control Drug
```



forcats — Renaming & Combining Levels

```
x <- factor(c("apple", "bear", "banana", "dates", "bear"))
```

```
# fct_recode() — rename levels
```

```
fct_recode(x, fruit = "apple", fruit = "banana",
           fruit = "dates", animal = "bear")
```

```
[1] fruit animal fruit fruit animal
```

```
Levels: fruit animal
```

```
# fct_lump() — lump small groups together
```

```
y <- factor(letters[rpois(100, 5)])
```

```
fct_lump(y, n = 5) # keep top 5, lump rest as "Other"
```

```
[1] Other e      Other Other Other d      c      c      f      c      e      e
[13] f      e      d      Other c      Other f      f      Other c      e      Other
[25] d      g      e      f      Other e      Other e      d      Other Other f
[37] g      g      Other g      e      Other g      g      e      e      e      d
[49] f      f      d      d      f      g      c      Other d      e      d      d
[61] f      c      c      Other d      e      Other Other d      Other Other e
[73] c      d      g      e      d      f      g      f      g      g      Other g
[85] c      Other c      f      e      e      g      f      Other g      d      Other
[97] e      c      e      g
```



Factors in Bioinformatics — Example

```
# Gene expression categories in a cancer study
expr_level <- c("Under-expressed", "Normal", "Over-expressed",
               "Over-expressed", "Normal", "Under-expressed",
               "Normal", "Normal", "Over-expressed", "Under-expressed")

( expr_f <- factor(expr_level,
                  levels = c("Under-expressed", "Normal", "Over-expressed"),
                  ordered = TRUE) )
```

```
[1] Under-expressed Normal          Over-expressed Over-expressed
[5] Normal          Under-expressed Normal          Normal
[9] Over-expressed Under-expressed

Levels: Under-expressed < Normal < Over-expressed
```

```
# Count how many genes in each category
table(expr_f)
```

```
expr_f
Under-expressed      Normal Over-expressed
              3             4             3
```

```
# Summary statistics by category
summary(expr_f)
```

Why Factors Matter

- ① **Statistical models:** `lm()` and `glm()` automatically create dummy variables from factors
- ② **Plot ordering:** `ggplot2` respects factor level order on axes
- ③ **Memory efficiency:** factors store integers internally, not full strings
- ④ **Data validation:** only values in `levels` are allowed



Tip

AI tip: Ask CodeBuddy: “My `ggplot` bar chart shows categories in wrong order — how do I reorder using `fct_relevel()`?”

What is a List?

A **list** is a container that can hold **different types** of objects — unlike vectors which require uniform types. Lists are R's most flexible data structure.

```
# A list can contain anything
my_list <- list(
  name      = "Gene TP53",
  chromosome = 17,
  is_oncogene = TRUE,
  expression = c(2.3, 4.5, 1.8, 3.2, 5.1),
  metadata   = list(study = "TCGA", cancer = "BRCA")
)
my_list
```

```
$name
[1] "Gene TP53"
```

```
$chromosome
[1] 17
```

```
$is_oncogene
[1] TRUE
```



List Structure

Lists are **recursive** — they can contain other lists:

```
str(my_list)      # inspect the structure
```

List of 5

```
$ name      : chr "Gene TP53"
$ chromosome : num 17
$ is_oncogene: logi TRUE
$ expression : num [1:5] 2.3 4.5 1.8 3.2 5.1
$ metadata   :List of 2
  ..$ study : chr "TCGA"
  ..$ cancer: chr "BRCA"
```

```
length(my_list)   # number of top-level elements
```

```
[1] 5
```

```
names(my_list)     # element names
```

```
[1] "name"          "chromosome"    "is_oncogene"  "expression"    "metadata"
```



List Structure, cont.

Lists are everywhere in bioinformatics R: - Statistical model outputs (`lm()`, `glm()`) - Genomic ranges (Bioconductor `GRanges`) - Seurat single-cell objects - JSON/API responses

Creating Lists

```
# Unnamed list  
list(1:5, letters[1:3], TRUE)
```

```
[[1]]  
[1] 1 2 3 4 5
```

```
[[2]]  
[1] "a" "b" "c"
```

```
[[3]]  
[1] TRUE
```

```
# Named list (preferred for readability)  
list(numbers = 1:5, chars = letters[1:3], flag = TRUE)
```

```
$numbers  
[1] 1 2 3 4 5
```

```
$chars  
[1] "a" "b" "c"
```

```
$flag
```



Creating Lists, cont.

```
# List from vectors of different lengths
list(
  patient_ids = c("P001", "P002", "P003"),
  ages        = c(45, 62, 38),
  diagnoses   = c("Healthy", "T2D", "Healthy"),
  medications = c("None", "Metformin"),      # shorter — no recycling!
  lab_results = data.frame(glucose = c(90, 140, 95),
                           insulin = c(5.2, 12.1, 4.8))
)
```

```
$patient_ids
[1] "P001" "P002" "P003"
```

```
$ages
[1] 45 62 38
```

```
$diagnoses
[1] "Healthy" "T2D"      "Healthy"
```

```
$medications
[1] "None"      "Metformin"
```

Accessing List Elements — \$ and [[]]

The \$ operator extracts a single named element:

```
my_list$name           # returns the element itself
```

```
[1] "Gene TP53"
```

```
my_list$expression     # returns the numeric vector
```

```
[1] 2.3 4.5 1.8 3.2 5.1
```

[[]] extracts by position or name:

```
my_list[[1]]           # first element
```

```
[1] "Gene TP53"
```

```
my_list[["name"]]      # by name (equivalent to $)
```

```
[1] "Gene TP53"
```



Accessing List Elements — [] vs [[]]

Single bracket [] returns a **sub-list**:

```
my_list[1]           # a list containing the first element
```

```
$name  
[1] "Gene TP53"
```

```
class(my_list[1])    # "list"
```

```
[1] "list"
```

Accessing List Elements — [] vs [[]], cont.

Double bracket [[]] returns the **element itself**:

```
my_list[[1]]          # the actual value
```

```
[1] "Gene TP53"
```

```
class(my_list[[1]])    # "character"
```

```
[1] "character"
```



Tip

Think of it like a train: [] gives you a train car (still a train), [[]] gives you what's inside the car.

Deep Access in Nested Lists

```
# Accessing nested elements  
my_list$metadata$study      # "TCGA"
```

```
[1] "TCGA"
```

```
my_list[["metadata"]][["cancer"]]  # "BRCA"
```

```
[1] "BRCA"
```

```
# Equivalent chaining  
my_list$metadata[[2]]           # also "BRCA" (2nd element of metadata)
```

```
[1] "BRCA"
```

Modifying Lists

```
# Add a new element
my_list$gene_length <- 19149 # base pairs

# Modify an existing element
my_list$expression <- c(3.1, 4.8, 2.0, 3.5, 5.5)

# Remove an element
my_list$metadata <- NULL

str(my_list)
```

List of 5

```
$ name      : chr "Gene TP53"
$ chromosome : num 17
$ is_oncogene: logi TRUE
$ expression : num [1:5] 3.1 4.8 2 3.5 5.5
$ gene_length: num 19149
```

Adding an element to a list is as simple as assigning to a new name with \$.



Lists in Bioinformatics — Practical Example

```
# Simulate a differential expression result
de_result <- list(
  gene = "BRCA1",
  log2FC = 2.34,
  p_value = 0.00042,
  fdr = 0.015,
  samples = c("Tumor1", "Tumor2", "Normal1", "Normal2")
)
```

```
# Extract p-value for downstream filtering
de_result$p_value
```

```
[1] 0.00042
```

```
# Check if gene is significant (FDR < 0.05)
de_result$fdr < 0.05
```

```
[1] TRUE
```



What is a Data Frame?

A **data frame** is a 2-dimensional table where each column can have a different type:

```
# Built-in example dataset
library(tidyverse)
library(kableExtra)
kbl(head(mpg), booktabs = TRUE)
```

manufacturer	model	displ	year	cyl	trans	drv	cty	hwy	fl	class
audi	a4	1.8	1999	4	auto(l5)	f	18	29	p	compact
audi	a4	1.8	1999	4	manual(m5)	f	21	29	p	compact
audi	a4	2.0	2008	4	manual(m6)	f	20	31	p	compact
audi	a4	2.0	2008	4	auto(av)	f	21	30	p	compact
audi	a4	2.8	1999	6	auto(l5)	f	16	26	p	compact
audi	a4	2.8	1999	6	manual(m5)	f	18	26	p	compact

- Each **column** is a vector of a single type
- Each **row** is one observation
- The most fundamental data structure for tabular data in R



Data Frame Properties

```
nrow(mpg)      # number of rows
```

```
[1] 234
```

```
ncol(mpg)      # number of columns
```

```
[1] 11
```

```
dim(mpg)       # both
```

```
[1] 234  11
```

```
str(mpg)       # structure overview
```

```
tibble [234 x 11] (S3: tbl_df/tbl/data.frame)
```

```
$ manufacturer: chr [1:234] "audi" "audi" "audi" "audi" ...
```

```
$ model       : chr [1:234] "a4" "a4" "a4" "a4" ...
```

```
$ displ      : num [1:234] 1.8 1.8 2 2 2.8 2.8 3.1 1.8 1.8 2 ...
```

```
$ year       : int [1:234] 1999 1999 2008 2008 1999 1999 2008 1999 1999 2008 ...
```

```
$ cyl        : int [1:234] 4 4 4 4 6 6 6 4 4 4 ...
```

```
$ trans      : chr [1:234] "auto(l5)" "manual(m5)" "manual(m6)" "auto(av)" ...
```

```
$ drv        : chr [1:234] "f" "f" "f" "f" ...
```

```
$ cty        : int [1:234] 18 21 20 21 16 18 18 18 16 20 ...
```



Data Frame Properties, cont.

```
# Quick peek
head(mpg, n = 3)      # first 3 rows
```

```
# A tibble: 3 x 11
```

	manufacturer	model	displ	year	cyl	trans	drv	cty	hwy	fl	class
	<chr>	<chr>	<dbl>	<int>	<int>	<chr>	<chr>	<int>	<int>	<chr>	<chr>
1	audi	a4	1.8	1999	4	auto(l5)	f	18	29	p	compa~
2	audi	a4	1.8	1999	4	manual(m5)	f	21	29	p	compa~
3	audi	a4	2	2008	4	manual(m6)	f	20	31	p	compa~

```
tail(mpg, n = 3)      # last 3 rows
```

```
# A tibble: 3 x 11
```

	manufacturer	model	displ	year	cyl	trans	drv	cty	hwy	fl	class
	<chr>	<chr>	<dbl>	<int>	<int>	<chr>	<chr>	<int>	<int>	<chr>	<chr>
1	volkswagen	passat	2.8	1999	6	auto(l5)	f	16	26	p	mids~
2	volkswagen	passat	2.8	1999	6	manual(m5)	f	18	26	p	mids~
3	volkswagen	passat	3.6	2008	6	auto(s6)	f	17	26	p	mids~



Creating Data Frames

```
# data.frame() function
( dat <- data.frame(
  gene      = c("BRCA1", "TP53", "EGFR", "MYC"),
  logFC     = c(2.3, -1.5, 0.8, 3.1),
  pvalue    = c(0.001, 0.04, 0.35, 0.0001),
  stringsAsFactors = FALSE    # keep characters as characters!
) )
```

	gene	logFC	pvalue
1	BRCA1	2.3	0.0010
2	TP53	-1.5	0.0400
3	EGFR	0.8	0.3500
4	MYC	3.1	0.0001

Warning

Prior to R 4.0, `data.frame()` automatically converted character columns to **factors**. Always use `stringsAsFactors = FALSE` unless you specifically need factors.

Data Frame Subsetting

Data frames use `[row, column]` for subsetting:

```
# Select rows 1-2
```

```
dat[1:2, ]
```

```
   gene logFC pvalue
1 BRCA1   2.3  0.001
2  TP53  -1.5  0.040
```

```
# Select a column (returns a vector by default)
```

```
dat[, "gene"]
```

```
[1] "BRCA1" "TP53" "EGFR" "MYC"
```

```
# Select by condition
```

```
dat[dat$pvalue < 0.05, ]      # significant genes only
```

```
   gene logFC pvalue
1 BRCA1   2.3 1e-03
2  TP53  -1.5 4e-02
4   MYC   3.1 1e-04
```

Data Frame — Adding & Removing Columns

```
# Add a column
dat$significant <- dat$pvalue < 0.05
dat
```

```
   gene logFC pvalue significant
1 BRCA1   2.3 0.0010          TRUE
2 TP53  -1.5 0.0400          TRUE
3 EGFR   0.8 0.3500         FALSE
4  MYC   3.1 0.0001          TRUE
```

```
# Add with cbind()
cbind(dat, rank = 1:4)
```

```
   gene logFC pvalue significant rank
1 BRCA1   2.3 0.0010          TRUE    1
2 TP53  -1.5 0.0400          TRUE    2
3 EGFR   0.8 0.3500         FALSE    3
4  MYC   3.1 0.0001          TRUE    4
```

```
# Remove a column
dat$significant <- NULL
dat
```

Tibble — The Modern Data Frame

A **tibble** is the tidyverse's upgraded data frame. Use `tibble()` or `tribble()`:

```
library(tibble)
```

```
# tibble() — by column
```

```
( tb <- tibble(
  gene  = c("BRCA1", "TP53", "EGFR"),
  logFC = c(2.3, -1.5, 0.8),
  sig    = c(TRUE, FALSE, TRUE)
) )
```

```
# A tibble: 3 x 3
```

```
  gene  logFC sig
  <chr> <dbl> <lgl>
1 BRCA1   2.3  TRUE
2 TP53  -1.5  FALSE
3 EGFR    0.8  TRUE
```

Tibble — The Modern Data Frame, cont.

```
# tribble() — by row (easier to read for small data)
tribble(
  ~gene, ~logFC, ~sig,
  "BRCA1", 2.3, TRUE,
  "TP53", -1.5, FALSE,
  "EGFR", 0.8, TRUE
)
```

```
# A tibble: 3 x 3
  gene  logFC sig
  <chr> <dbl> <lgl>
1 BRCA1   2.3  TRUE
2 TP53  -1.5  FALSE
3 EGFR    0.8  TRUE
```

Tibble vs Data Frame — Key Differences

1. Sequential column evaluation

```
# Tibble: columns evaluated left-to-right
```

```
tibble(x = 1:5, y = x^2)
```

```
# A tibble: 5 x 2
```

	x	y
	<int>	<dbl>
1	1	1
2	2	4
3	3	9
4	4	16
5	5	25

```
# data.frame: this fails!
```

```
# data.frame(x = 1:5, y = x^2)
```

Tibble vs Data Frame — Key Differences, cont.

2. Subsetting never simplifies

```
df <- data.frame(x = 1:3, y = 3:1)
tb <- tibble(x = 1:3, y = 3:1)
```

```
# data.frame: single column becomes a vector
class(df[, 1])          # "integer"
```

```
[1] "integer"
```

```
# tibble: always stays a tibble
```

```
class(tb[, 1])          # "tbl_df" "tbl" "data.frame"
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

Tibble vs Data Frame — More Differences

3. Recycling rules

```
# data.frame: recycles freely (6 is multiple of 2)  
data.frame(a = 1:6, b = LETTERS[1:2])      # works silently!
```

```
  a b  
1 1 A  
2 2 B  
3 3 A  
4 4 B  
5 5 A  
6 6 B
```

Tibble vs Data Frame — More Differences, cont.

```
# tibble: only length-1 or exact match
# tibble(a = 1:6, b = LETTERS[1:2])      # ERROR!
tibble(a = 1:6, b = 1)                   # length-1: OK
```

```
# A tibble: 6 x 2
```

```
      a      b
  <int> <dbl>
1     1     1
2     2     1
3     3     1
4     4     1
5     5     1
6     6     1
```

4. Partial matching

```
df <- data.frame(abc = 1:3)
df$ab      # partial match — dangerous!
```

```
[1] 1 2 3
```

```
tb <- tibble(abc = 1:3)
# tb$a      # NULL + warning — safer!
```

Converting Between Types

```
# data.frame → tibble
as_tibble(iris)
```

```
# A tibble: 150 x 5
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<fct>
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa

```
# i 140 more rows
```



Converting Between Types, cont.

```
# tibble → data.frame
as.data.frame(as_tibble(iris)[1:5, ])
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa



Tip

In this course, we prefer **tibbles** for everyday work. Convert to `data.frame` only when a function requires it.



Working with tibbles — Add Rows & Columns

```
tb <- tibble(gene = character(), logFC = double())
```

```
# Add rows one by one
```

```
tb <- add_row(tb, gene = "BRCA1", logFC = 2.3)
```

```
tb <- add_row(tb, gene = "TP53", logFC = -1.5)
```

```
tb
```

```
# A tibble: 2 x 2
```

```
  gene logFC
```

```
  <chr> <dbl>
```

```
1 BRCA1    2.3
```

```
2 TP53   -1.5
```

```
# Add a column
```

```
tb <- add_column(tb, significant = c(TRUE, FALSE))
```

```
tb
```

```
# A tibble: 2 x 3
```

```
  gene logFC significant
```

```
  <chr> <dbl> <lgl>
```

```
1 BRCA1    2.3 TRUE
```

```
2 TP53   -1.5 FALSE
```



Working with tibbles — Add Rows & Columns, cont.

```
# Combine tibbles by row  
bind_rows(tb, tb)
```

```
# A tibble: 4 x 3  
  gene  logFC significant  
  <chr> <dbl> <lgl>  
1 BRCA1   2.3    TRUE  
2 TP53  -1.5    FALSE  
3 BRCA1   2.3    TRUE  
4 TP53  -1.5    FALSE
```

attach(), with(), and within()

```
# attach: access columns directly (use sparingly!)
attach(iris)
head(Sepal.Length, 3)
```

```
[1] 5.1 4.9 4.7
```

```
detach(iris)
```

```
# with: safer temporary access
with(iris, head(Sepal.Length, 3))
```

```
[1] 5.1 4.9 4.7
```

```
# within: modify multiple columns at once
head(within(airquality, {
  lOzone <- log(Ozone)
  cTemp  <- round((Temp - 32) * 5 / 9, 1)
}), 3)
```

	Ozone	Solar.R	Wind	Temp	Month	Day	cTemp	lOzone
1	41	190	7.4	67	5	1	19.4	3.713572
2	36	118	8.0	72	5	2	22.2	3.583519
3	12	149	12.6	74	5	3	23.3	2.484907



Reading Data with readr

readr (part of tidyverse) provides fast, consistent file reading:

Function	Purpose
<code>read_csv()</code>	Comma-separated files
<code>read_tsv()</code>	Tab-separated files
<code>read_delim()</code>	Custom delimiters
<code>read_fwf()</code>	Fixed-width files
<code>read_table()</code>	Whitespace-separated columns
<code>read_log()</code>	Web server logs

```
library(tidyverse) # readr is included  
# or: library(readr)
```

Reading CSV Files

```
# Basic reading
myiris <- read_csv("data/talk03/iris.csv")
myiris
```

```
# A tibble: 150 x 5
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa
10	4.9	3.1	1.5	0.1	setosa

```
# i 140 more rows
```



Reading CSV Files, cont.

With explicit column types:

```
read_csv("data/talk03/iris.csv", col_types = cols(
  Sepal.Length = col_double(),
  Sepal.Width  = col_double(),
  Petal.Length = col_double(),
  Petal.Width  = col_double(),
  Species      = col_character()
))
```

A tibble: 150 x 5

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
	<dbl>	<dbl>	<dbl>	<dbl>	<chr>
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa
7	4.6	3.4	1.4	0.3	setosa
8	5	3.4	1.5	0.2	setosa
9	4.4	2.9	1.4	0.2	setosa



Column Type Specifications

<code>col_*()</code>	Type
<code>col_double()</code>	Double-precision numbers
<code>col_integer()</code>	Integers
<code>col_character()</code>	Text strings
<code>col_logical()</code>	TRUE/FALSE
<code>col_factor()</code>	Factors (categorical)
<code>col_date()</code>	Dates
<code>col_datetime()</code>	Date-times
<code>col_skip()</code>	Skip this column
<code>col_guess()</code>	Let readr guess

Use `col_types = cols(...)` to ensure correct parsing, especially for date columns or numeric columns with special formatting.

Writing Data

Function	Output format
<code>write_csv()</code>	Comma-separated
<code>write_tsv()</code>	Tab-separated
<code>write_delim()</code>	Custom delimiter
<code>write_excel_csv()</code>	CSV optimised for Excel
<code>write_rds()</code>	R binary format
<code>write_lines()</code>	One string per line

```
# Write iris to CSV
write_csv(iris, "iris_output.csv")

# Write to TSV
write_tsv(iris, "iris_output.tsv")
```

Reading Other Formats

Package	Format
readxl	Excel (.xls, .xlsx)
haven	SPSS, Stata, SAS
jsonlite	JSON
xml2	XML
DBI	Databases (SQLite, PostgreSQL)
httr	Web APIs
rvest	HTML (web scraping)

For bioinformatics: use `read_csv()/read_tsv()` for most flat files; `readxl` for Excel files from collaborators; `httr` for NCBI/Ensembl APIs.

What is Tidy Data?

Tidy data is a standardized format where:

- ① Each **variable** has its own column
- ② Each **observation** has its own row
- ③ Each **value** has its own cell



Tip

Tidy data makes it easy to filter, group, summarize, and plot. It's the foundation of the entire tidyverse workflow. (See R4DS by Wickham & Grolemund for visual examples.)

Tidy vs Untidy — Example

Untidy (wide format — common in Excel):

Column headers are values (genes), not variable names

```
untidy <- tibble(
  patient = c("P1", "P2", "P3"),
  BRCA1   = c(2.3, 1.8, 3.1),
  TP53    = c(-1.2, -0.8, 0.5),
  EGFR    = c(0.5, 1.1, -0.3)
)
untidy
```

A tibble: 3 x 4

	patient	BRCA1	TP53	EGFR
	<chr>	<dbl>	<dbl>	<dbl>
1	P1	2.3	-1.2	0.5
2	P2	1.8	-0.8	1.1
3	P3	3.1	0.5	-0.3

Tidy vs Untidy — Example, cont.

Tidy (long format):

```
library(tidyr)
tidy <- pivot_longer(untidy, cols = c(BRCA1, TP53, EGFR),
                     names_to = "gene", values_to = "expression")
tidy
```

A tibble: 9 x 3

	patient	gene	expression
	<chr>	<chr>	<dbl>
1	P1	BRCA1	2.3
2	P1	TP53	-1.2
3	P1	EGFR	0.5
4	P2	BRCA1	1.8
5	P2	TP53	-0.8
6	P2	EGFR	1.1
7	P3	BRCA1	3.1
8	P3	TP53	0.5
9	P3	EGFR	-0.3

Why Tidy Data?

- 1 **dplyr** and **ggplot2** are designed for tidy data
- 2 Easier to filter, group, and summarize
- 3 Consistent mental model — you always know where to find things
- 4 The foundation of the entire tidyverse workflow



Tip

“Tidy datasets are all alike, but every messy dataset is messy in its own way.” — Hadley Wickham

Working with Dates — lubridate

Biological data often involves dates: sample collection dates, patient birth dates, follow-up visits. The **lubridate** package makes date handling intuitive.

```
library(lubridate)
```

```
# Parse dates from strings
```

```
ymd("2024-03-15")           # year-month-day
```

```
[1] "2024-03-15"
```

```
mdy("March 15, 2024")      # month-day-year
```

```
[1] "2024-03-15"
```

```
dmy("15-03-2024")         # day-month-year
```

```
[1] "2024-03-15"
```

```
# Parse date-times
```

```
ymd_hms("2024-03-15 14:30:00")
```

```
[1] "2024-03-15 14:30:00 UTC"
```



Date Extraction & Arithmetic

```
today() # current date
```

```
[1] "2026-05-27"
```

```
now() # current date-time
```

```
[1] "2026-05-27 20:48:40 CST"
```

```
# Extract components
```

```
date <- ymd("2024-03-15")
```

```
year(date) # 2024
```

```
[1] 2024
```

```
month(date) # 3
```

```
[1] 3
```

```
day(date) # 15
```

```
[1] 15
```

```
wday(date, label = TRUE) # Fri (weekday name)
```

```
[1] Fri
```



Date Extraction & Arithmetic, cont.

```
# Date arithmetic  
date + days(7)           # one week later
```

```
[1] "2024-03-22"
```

```
date - days(30)          # 30 days earlier
```

```
[1] "2024-02-14"
```

Today's Summary

Topic	Key Concepts
Factors	<code>factor()</code> , <code>levels</code> , <code>ordered</code> , <code>forcats</code> (<code>fct_relevel</code> , <code>fct_recode</code> , <code>fct_lump</code>)
Lists	Flexible containers, <code>\$</code> / <code>[[]]</code> vs <code>[]</code> , nested access
Data Frames	<code>data.frame()</code> , <code>nrow/ncol</code> , subsetting <code>[row, col]</code> , <code>rbind/cbind</code>
Tibbles	<code>tibble()/tribble()</code> , no simplification, strict recycling, sequential evaluation
File I/O	<code>read_csv</code> , <code>read_tsv</code> , <code>write_csv</code> , <code>col_types</code>
Tidy Data	Variables = columns, observations = rows, <code>pivot_longer</code>
lubridate	<code>ymd()</code> , <code>mdy()</code> , <code>today()</code> , date arithmetic

Homework

- 1 Open Homework_AI_powered_R/talk03-homework.qmd in your course repository
- 2 Complete all exercises — use **CodeBuddy AI** for help!
- 3 Render to PDF
- 4 Submit as instructed on the course platform



Tip

AI tips for this talk: - “Create a factor with levels in this specific order: ...” - “How do I extract a nested element from a list in R?” - “Why does my read_csv parse this column as character instead of numeric?” - “Convert this wide gene expression table to tidy long format”

Next Talk

Talk 04 — Data Wrangling with tidyverse

- The pipe operator (`%>%` / `|>`)
- dplyr core verbs: `filter`, `select`, `mutate`, `arrange`, `summarise` + `group_by`
- tidyr: `pivot_longer`, `pivot_wider`, `separate`, `unite`
- Joins: `left_join`, `inner_join`, `full_join`